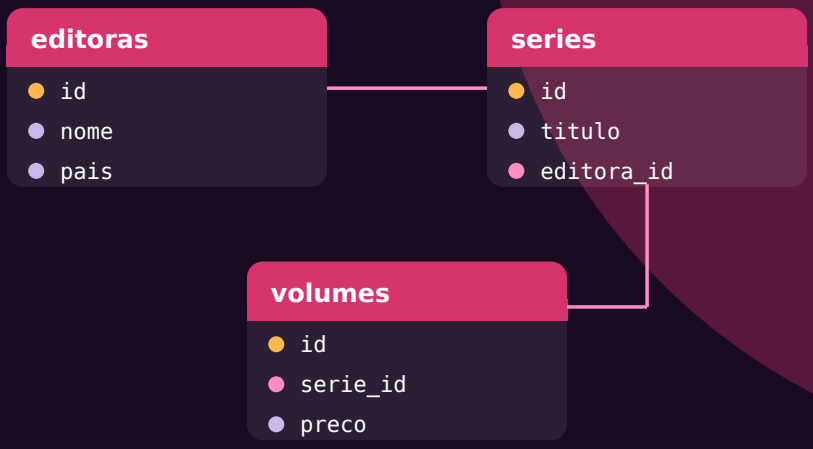


MATERIAL DE ESTUDO · LABORATÓRIO 2 · 7 FASES, 44 MISSÕES

SQL Lab 2

Monte o banco da MangáVerso

Passo a passo para criar bancos e tabelas do zero: ENUM, CHECK, chaves estrangeiras, relação N:N, UPDATE seguro, transações, views e UNION, com MER e DER automáticos

MySQL · MER · DER · Views · Transações

```
CREATE TABLE series ( ... FOREIGN KEY (editora_id) REFERENCES editoras(id) );
```

Israel Pinheiro Lemos

Professor · Arquiteto de Software Full Stack

Sumário

Sete fases, uma por capítulo, na mesma ordem das missões do laboratório.

1. Abrindo a loja: o banco e as tabelas do catálogo 6

2. Enchendo as prateleiras: INSERT com várias linhas 11

3. Consultando o catálogo 15

4. Clientes e pedidos: relação de muitos para muitos 23

5. Mudando com segurança 30

6. Relatórios guardados: Views e UNION 34

7. Desafio final: monte o seu banco 38

Cola de comandos 42

Lendo o MER e o DER 44

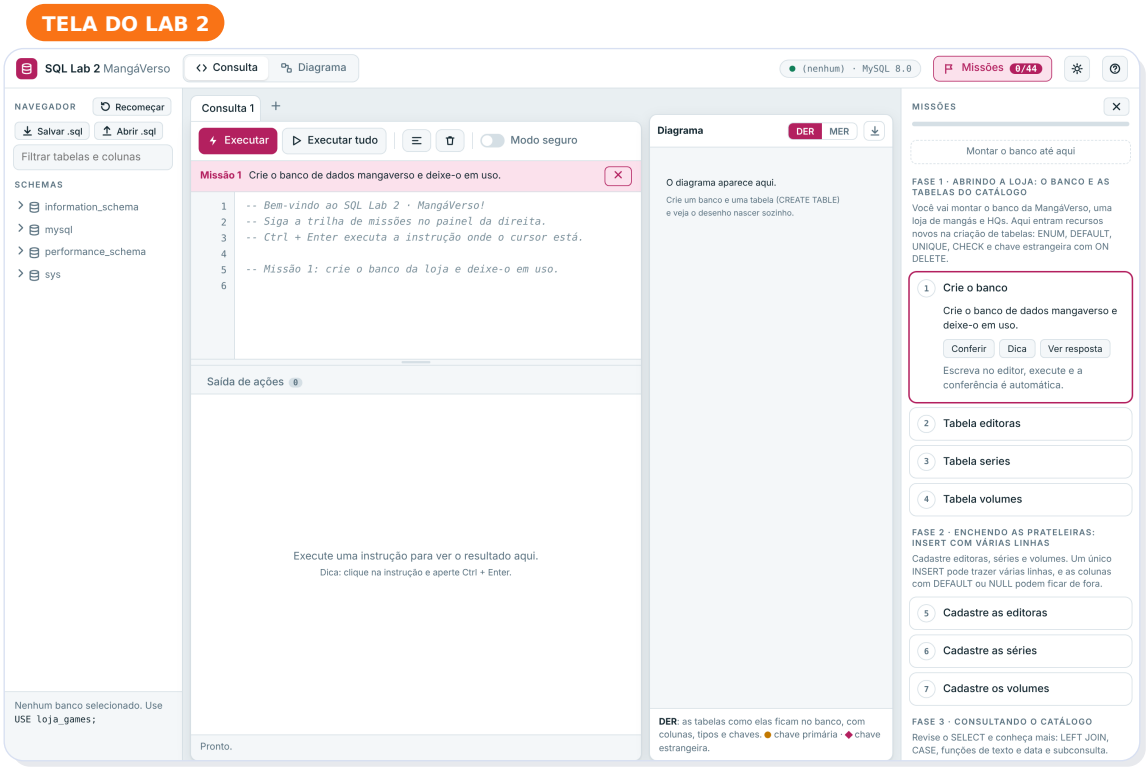
Erros comuns e o que fazer 46

Como estudar com esta apostila

Este é o **Laboratório 2**, a continuação do primeiro (a biblioteca da escola). Lá você aprendeu a criar uma tabela, inserir, consultar e juntar. Aqui você vai **construir bancos inteiros**, do zero, com várias tabelas ligadas, usando comandos novos na sequência. O tema agora é a **MangáVerso**, uma loja de mangás, HQs e webtoons.

A apostila e o laboratório andam juntos, na mesma ordem. Para cada **missão** do painel da direita há um **passo** aqui, que explica o conceito, mostra o comando e diz o que você deve ver.

A tela do laboratório



1. **Navegador**: os bancos e as tabelas que você criar aparecem aqui.
2. **Editor**: onde você escreve o SQL. Ctrl + Enter executa a instrução do cursor.
3. **Faixa da missão**: mostra o que fazer agora.
4. **Diagrama ao vivo**: DER ou MER, desenhado sozinho a cada tabela que você cria.
5. **Trilha de missões**: 44 missões em 7 fases, com Dica, Ver resposta e Conferir.

O ritmo de cada missão

1. Leia o enunciado no painel (ou aqui, no quadro “Faça junto”).
2. Escreva o comando no editor. Tente sozinho antes de olhar a dica.
3. Execute com **Ctrl + Enter** (uma instrução) ou **Ctrl + Shift + Enter** (tudo).

4. Olhe o resultado e o diagrama. A missão é conferida automaticamente e ganha o ✓ verde.
5. Se faltar algo, o painel diz exatamente o quê. O botão **Conferir** testa de novo.

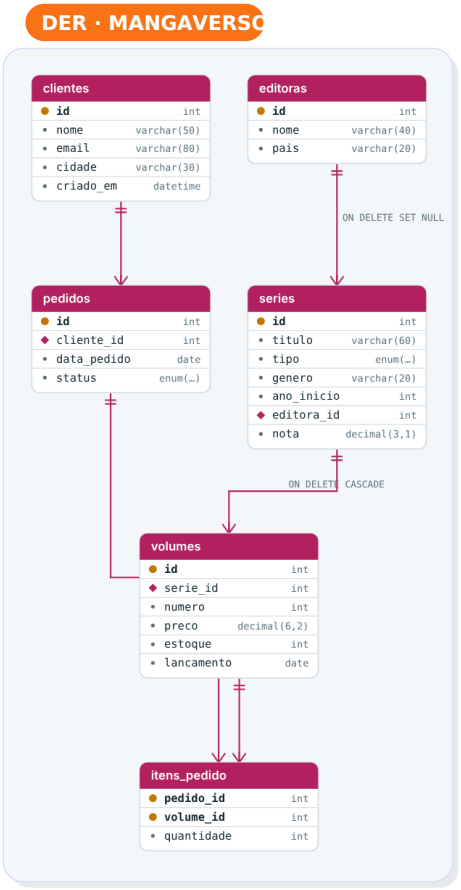
> **Dica.** Se você pular uma missão e o banco ficar incompleto, o botão **Montar o banco até aqui** (no painel) apaga tudo e refaz o banco com as respostas das missões anteriores à atual, para você continuar sem travar.

> **Importante.** O seu progresso e o seu banco ficam guardados **neste navegador**. O botão **Recomeçar** apaga tudo e volta ao início. Para levar o banco para outro lugar, use **Salvar .sql** e depois **Abrir .sql**.

O mapa do projeto

Ao final da fase 6 o banco da MangáVerso terá estas seis tabelas e duas views. Use este desenho como mapa: sempre que se perder, volte aqui.

DER: as tabelas como ficam no banco



- **editoras → series**: uma editora publica várias séries (a série pode ficar sem editora).
- **series → volumes**: uma série tem vários volumes.
- **clientes → pedidos**: um cliente faz vários pedidos.
- **pedidos ↔ volumes**: relação N:N, resolvida pela tabela **itens_pedido**.

CAPÍTULO 1

Abrindo a loja: o banco e as tabelas do catálogo

Criar o banco, as tabelas do catálogo e ler o diagrama que nasce sozinho.

VOCÊ VAI APRENDER

- ✓ CREATE DATABASE e USE
- ✓ ENUM, DEFAULT, UNIQUE, NOT NULL e CHECK
- ✓ Chave estrangeira com ON DELETE
- ✓ Ler o DER e o MER ao lado do editor

Você vai montar o banco da **MangáVerso**, uma loja de mangás, HQs e webtoons. Cada missão deste capítulo cria uma peça do catálogo, e o diagrama ao lado do editor mostra o resultado na hora. Escreva cada comando no editor, execute com **Ctrl + Enter** e confira se a missão foi marcada como resolvida no painel da direita.

1 Crie o banco

Faça junto. Missão 1. Crie o banco de dados mangaverso e deixe-o em uso.

Todo projeto começa com o banco de dados, o "armário" que vai guardar as tabelas. **CREATE DATABASE** cria o armário e **USE** diz ao MySQL: "a partir de agora, trabalhe dentro dele". Sem o USE, as próximas tabelas não sabem onde nascer.

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`CREATE DATABASE mangaverso;`	cria um banco novo chamado mangaverso
`USE mangaverso;`	escolhe esse banco como o banco de trabalho

SQL · missão 1

```sql
CREATE DATABASE mangaverso;
USE mangaverso;
```

Dica. Olhe o navegador à esquerda: o banco **mangaverso** aparece na lista e o selo no alto da tela mostra o banco em uso.

2 Tabela editoras

Faça junto. Missão 2. Crie a tabela editoras com: id (inteiro, automático, chave primária), nome (texto de até 40, obrigatório e único) e pais (texto de até 20, valor padrão 'Japão').

Agora a primeira tabela: **editoras**, as empresas que publicam as séries. Aqui entram três ideias novas: **UNIQUE** (não aceita dois valores iguais), **NOT NULL** (campo obrigatório) e **DEFAULT** (valor usado quando você não informa nada).

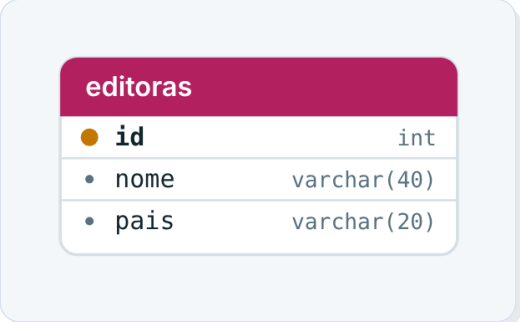
CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`id INT AUTO_INCREMENT PRIMARY KEY`	número inteiro que o banco conta sozinho (1, 2, 3...) e identifica cada linha
`nome VARCHAR(40) NOT NULL UNIQUE`	texto de até 40 letras, obrigatório e sem repetição
`pais VARCHAR(20) DEFAULT 'Japão'`	se o país não for informado, vale 'Japão'

SQL · missão 2

```sql
CREATE TABLE editoras (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(40) NOT NULL UNIQUE,
  pais VARCHAR(20) DEFAULT 'Japão'
);
```

NO LAB



Importante. Assim que a tabela nasce, o **diagrama ao lado do editor** desenha a caixa **editoras**, com a chave primária marcada por uma bolinha dourada.

3 Tabela series

Faça junto. Missão 3. Crie a tabela series com: id (automático, chave primária), titulo (texto de até 60, obrigatório), tipo (ENUM com 'mangá', 'HQ' e 'webtoon', obrigatório), genero (texto de até 20), ano_inicio (inteiro) e editora_id (inteiro). A editora_id é chave estrangeira de editoras e, se a editora for apagada, vira NULL (ON DELETE SET NULL).

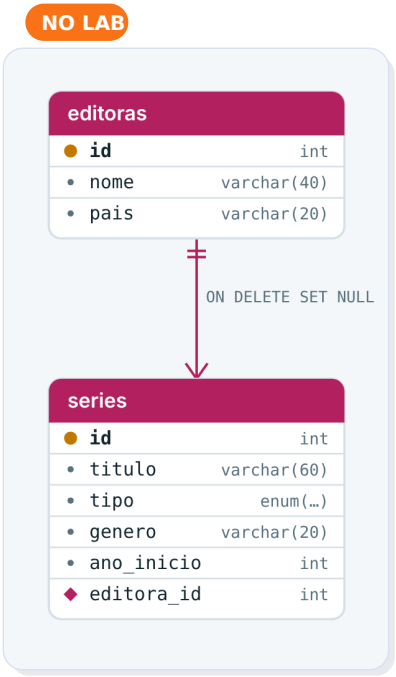
A tabela **series** guarda cada mangá, HQ ou webtoon. O campo **tipo** usa **ENUM**: uma lista fechada de valores permitidos. Já **editora_id** é uma **chave estrangeira**: ela aponta para o **id** de uma editora e cria a ligação entre as duas tabelas.

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`tipo ENUM('mangá','HQ','webtoon') NOT NULL`	só aceita um desses três valores
`editora_id INT`	guarda o número da editora (pode ficar vazio)
`FOREIGN KEY (editora_id) REFERENCES editoras(id)`	obriga o valor a existir na tabela editoras
`ON DELETE SET NULL`	se a editora for apagada, o editora_id da série vira NULL em vez de dar erro

SQL · missão 3

```sql
CREATE TABLE series (
  id INT AUTO_INCREMENT PRIMARY KEY,
  titulo VARCHAR(60) NOT NULL,
  tipo ENUM('mangá','HQ','webtoon') NOT NULL,
  genero VARCHAR(20),
  ano_inicio INT,
  editora_id INT,
  FOREIGN KEY (editora_id) REFERENCES editoras(id) ON DELETE SET NULL
);
```



Dica. Veja no diagrama: surgiu uma linha ligando **series** a **editoras**, com o "pé de galinha" no lado de series (muitas séries para uma editora) e a legenda do ON DELETE.

4 Tabela volumes

Faça junto. Missão 4. Crie a tabela volumes com: id (automático, chave primária), serie_id (inteiro, obrigatório), numero (inteiro, obrigatório), preco (DECIMAL(6,2) com CHECK preco > 0), estoque (inteiro, padrão 0) e lancamento (DATE). A dupla serie_id + numero não pode repetir (UNIQUE). A serie_id é chave estrangeira de series com ON DELETE CASCADE.

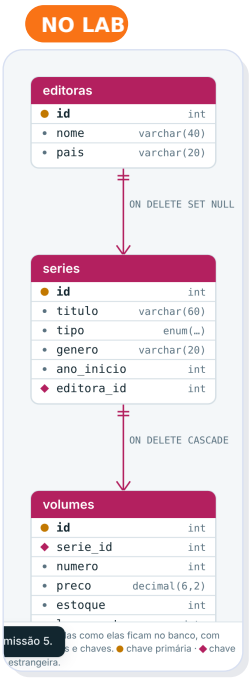
A tabela **volumes** é o que a loja realmente vende: cada exemplar numerado de uma série. Duas novidades: o **CHECK** (uma regra que o dado precisa cumprir, aqui preço maior que zero) e o **UNIQUE com duas colunas**, que impede cadastrar duas vezes o mesmo número da mesma série.

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`preco DECIMAL(6,2) CHECK (preco > 0)`	até 6 dígitos, 2 depois da vírgula, e nunca zero ou negativo
`estoque INT DEFAULT 0`	se ninguém informar, o estoque começa em 0
`UNIQUE (serie_id, numero)`	a combinação série + número não pode se repetir
`ON DELETE CASCADE`	apagou a série, os volumes dela vão embora junto

SQL · missão 4

```sql
CREATE TABLE volumes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  serie_id INT NOT NULL,
  numero INT NOT NULL,
  preco DECIMAL(6,2) CHECK (preco > 0),
  estoque INT DEFAULT 0,
  lancamento DATE,
  UNIQUE (serie_id, numero),
  FOREIGN KEY (serie_id) REFERENCES series(id) ON DELETE CASCADE
);
```



Pare e pense. Qual a diferença entre **SET NULL** (da tabela series) e **CASCADE** (desta)? No primeiro o filho continua existindo, sem pai; no segundo o filho desaparece com o pai.

Resumo do capítulo

- Um banco guarda tabelas; USE escolhe o banco de trabalho.
- UNIQUE impede repetidos, NOT NULL obriga o preenchimento, DEFAULT preenche sozinho e CHECK impõe uma regra.
- FOREIGN KEY liga duas tabelas; ON DELETE decide o que acontece quando o “pai” é apagado.

CAPÍTULO 2

Enchendo as prateleiras: INSERT com várias linhas

Povoar as tabelas com INSERT de várias linhas, respeitando as chaves estrangeiras.

VOCÊ VAI APRENDER

- ✓ INSERT com várias linhas
- ✓ Deixar colunas com DEFAULT de fora
- ✓ NULL e formato de datas
- ✓ A ordem certa: pai antes do filho

Tabelas vazias não respondem a nada. Neste capítulo você cadastra editoras, séries e volumes. Há uma ordem a respeitar: como a chave estrangeira exige que o “pai” exista, cadastre **editoras**, depois **series**, depois **volumes**.

5 Cadastre as editoras

Faça junto. Missão 5. Cadastre 4 editoras: Shueisha e Kodansha (deixe o país no valor padrão), Marvel Comics (EUA) e Maurício de Sousa Produções (Brasil).

Com as tabelas prontas, é hora de povoar. Um INSERT pode cadastrar várias linhas de uma vez, separando cada grupo de valores por vírgula. Quando uma coluna tem DEFAULT, basta **não citá-la** na lista de colunas.

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`INSERT INTO editoras (nome) VALUES ...`	lista só a coluna nome; o pais assume 'Japão'
`('Shueisha'), ('Kodansha')`	duas linhas em um único comando

SQL · missão 5

```sql
INSERT INTO editoras (nome) VALUES ('Shueisha'), ('Kodansha');
INSERT INTO editoras (nome, pais) VALUES ('Marvel Comics', 'EUA'),
    ('Maurício de Sousa Produções', 'Brasil');
```

Erro comum. Se aparecer o erro 1062 (Duplicate entry), você tentou cadastrar uma editora que já existe: o UNIQUE de **nome** fez o trabalho dele.

6 Cadastre as séries

Faça junto. Missão 6. Cadastre as 8 séries abaixo em um único INSERT. A Tower of God ainda não tem editora (NULL).

DADOS PARA CADASTRAR

título	tipo	gênero	ano	editora_id
Dragon Ball	mangá	Luta	1984	1
One Piece	mangá	Aventura	1997	1
Naruto	mangá	Ninja	1999	1
Jujutsu Kaisen	mangá	Sobrenatural	2018	1
Attack on Titan	mangá	Ação	2009	2
Homem-Aranha	HQ	Super-herói	1962	3
Turma da Mônica Jovem	HQ	Aventura	2008	4
Tower of God	webtoon	Fantasia	2010	NULL

Cadastre as 8 séries em **um único INSERT**. Repare no último valor: **NULL** significa “sem valor”. A Tower of God ainda não tem editora, então escrevemos NULL (sem aspas) no lugar do número.

SQL · missão 6

```sql
INSERT INTO series (titulo, tipo, genero, ano_inicio, editora_id) VALUES
  ('Dragon Ball', 'mangá', 'Luta', 1984, 1),
  ('One Piece', 'mangá', 'Aventura', 1997, 1),
  ('Naruto', 'mangá', 'Ninja', 1999, 1),
  ('Jujutsu Kaisen', 'mangá', 'Sobrenatural', 2018, 1),
  ('Attack on Titan', 'mangá', 'Ação', 2009, 2),
  ('Homem-Aranha', 'HQ', 'Super-herói', 1962, 3),
  ('Turma da Mônica Jovem', 'HQ', 'Aventura', 2008, 4),
  ('Tower of God', 'webtoon', 'Fantasia', 2010, NULL);
```

Cuidado. Textos vão entre **aspas simples**. Números e NULL ficam sem aspas. A ordem dos valores precisa seguir a ordem das colunas listadas.

7 Cadastre os volumes

Faça junto. Missão 7. Cadastre os 14 volumes abaixo (serie_id, numero, preco, estoque, lancamento) em um único INSERT.

DADOS PARA CADASTRAR

série	nº	preço	estoque	lançamento
1	1	29.90	12	2020-03-10
1	2	29.90	8	2020-04-10
1	3	29.90	0	2020-05-10
2	1	32.90	15	2021-01-15
2	2	32.90	10	2021-02-15
3	1	27.90	20	2019-06-01
3	2	27.90	5	2019-07-01
4	1	34.90	9	2022-09-20
5	1	39.90	6	2021-11-05
6	1	49.90	4	2023-02-14
6	2	49.90	0	2023-03-14
7	1	24.90	18	2022-05-05
8	1	44.90	7	2023-08-08
8	2	44.90	3	2023-09-08

Agora os volumes. Atenção às datas: o formato do MySQL é **'AAAA-MM-DD'**, sempre entre aspas. Os valores de série_id só funcionam porque as séries de 1 a 8 já existem: é a chave estrangeira conferindo.

SQL · missão 7

```sql
INSERT INTO volumes (serie_id, numero, preco, estoque, lancamento) VALUES
  (1, 1, 29.90, 12, '2020-03-10'),
  (1, 2, 29.90, 8, '2020-04-10'),
  (1, 3, 29.90, 0, '2020-05-10'),
  (2, 1, 32.90, 15, '2021-01-15'),
  (2, 2, 32.90, 10, '2021-02-15'),
  (3, 1, 27.90, 20, '2019-06-01'),
  (3, 2, 27.90, 5, '2019-07-01'),
  (4, 1, 34.90, 9, '2022-09-20'),
  (5, 1, 39.90, 6, '2021-11-05'),
  (6, 1, 49.90, 4, '2023-02-14'),
  (6, 2, 49.90, 0, '2023-03-14'),
  (7, 1, 24.90, 18, '2022-05-05'),
  (8, 1, 44.90, 7, '2023-08-08'),
  (8, 2, 44.90, 3, '2023-09-08');
```

Erro comum. Erro 1452 (Cannot add or update a child row): você usou um serie_id que não existe. Cadastre sempre o “pai” antes do “filho”.

Resumo do capítulo

- Várias linhas em um INSERT: separe os grupos por vírgula.
- Textos e datas entre aspas simples; NULL e números sem aspas.
- Cadastre sempre o pai antes do filho.

CAPÍTULO 3

Consultando o catálogo

Revisar o SELECT e aprender LEFT JOIN, CASE, funções e subconsulta.

3

VOCÊ VAI APRENDER

- ✓ WHERE, ORDER BY, BETWEEN, LIKE e IS NULL
- ✓ JOIN e LEFT JOIN
- ✓ GROUP BY, HAVING, COUNT e SUM
- ✓ CASE, UPPER, CONCAT, YEAR e subconsulta

Agora o catálogo está cheio e é hora de perguntar coisas a ele. Cada missão é uma pergunta; a sua consulta é conferida pelo **resultado** que devolve, então há mais de um jeito certo de escrever. Execute e compare com o resultado esperado que aparece em cada passo.

8 Só os mangás

Faça junto. Missão 8. Mostre todas as colunas das séries do tipo mangá.

Começa a fase das consultas. **WHERE** filtra as linhas. Para comparar texto, use aspas simples: **tipo = 'mangá'**.

SQL · missão 8

```
SELECT * FROM series WHERE tipo = 'mangá';
```

RESULTADO ESPERADO

id	titulo	tipo	genero	ano_inicio	editora_id
1	Dragon Ball	mangá	Luta	1984	1
2	One Piece	mangá	Aventura	1997	1
3	Naruto	mangá	Ninja	1999	1
4	Jujutsu Kaisen	mangá	Sobrenatural	2018	1
5	Attack on Titan	mangá	Ação	2009	2

9 Séries recentes

Faça junto. Missão 9. Mostre titulo e ano_inicio das séries que começaram depois de 2000, da mais nova para a mais antiga.

ORDER BY coluna DESC ordena do maior para o menor (DESC) ou do menor para o maior (ASC, que é o padrão). Os filtros numéricos usam **>**, **<**, **>=**, **<=** e **=**.

```
SQL · missão 9
SELECT titulo, ano_inicio FROM series WHERE ano_inicio > 2000 ORDER BY ano_inicio DESC;
```

RESULTADO ESPERADO

titulo	ano_inicio
Jujutsu Kaisen	2018
Tower of God	2010
Attack on Titan	2009
Turma da Mônica Jovem	2008

10 Preço na faixa

Faça junto. Missão 10. Mostre serie_id, numero e preco dos volumes com preço entre 30 e 45.

BETWEEN a AND b é um atalho para “maior ou igual a a e menor ou igual a b”. Os dois extremos entram no resultado.

```
SQL · missão 10
SELECT serie_id, numero, preco FROM volumes WHERE preco BETWEEN 30 AND 45;
```

RESULTADO ESPERADO

serie_id	numero	preco
2	1	32.90
2	2	32.90
4	1	34.90
5	1	39.90
8	1	44.90
8	2	44.90

11 Título começa com T

Faça junto. Missão 11. Mostre o titulo das séries cujo título começa com a letra T.

LIKE procura por padrão em textos. O **%** significa “qualquer coisa, de qualquer tamanho”. **'T%'** = começa com T. **'%man%'** = tem “man” em qualquer parte.

SQL · missão 11

```
SELECT titulo FROM series WHERE titulo LIKE 'T%';
```

RESULTADO ESPERADO

titulo
Turma da Mônica Jovem
Tower of God

12 Série sem editora

Faça junto. Missão 12. Mostre o titulo das séries que ainda não têm editora cadastrada.

Para testar valores vazios, use **IS NULL** (ou IS NOT NULL). Nunca escreva **= NULL**: NULL não é um valor, é a ausência dele, e a comparação com = nunca dá verdadeiro.

SQL · missão 12

```
SELECT titulo FROM series WHERE editora_id IS NULL;
```

RESULTADO ESPERADO

titulo
Tower of God

13 Série e editora

Faça junto. Missão 13. Mostre o titulo de cada série ao lado do nome da editora (apelido editora). Só entram séries que têm editora.

O **JOIN** costura duas tabelas usando a chave estrangeira. Os apelidos **s** e **e** encurtam a escrita. O JOIN simples só devolve séries que **têm** editora: a Tower of God fica de fora.

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`FROM series s`	tabela principal, com apelido s
`JOIN editoras e`	tabela ligada, com apelido e
`ON s.editora_id = e.id`	a regra da ligação: chave estrangeira = chave primária

```
SQL · missão 13
SELECT s.titulo, e.nome AS editora FROM series s JOIN editoras e ON s.editora_id = e.id;
```

RESULTADO ESPERADO

titulo	editora
Dragon Ball	Shueisha
One Piece	Shueisha
Naruto	Shueisha
Jujutsu Kaisen	Shueisha
Attack on Titan	Kodansha
Homem-Aranha	Marvel Comics
Turma da Mônica Jovem	Maurício de Sousa Produções

14 Todas as séries (LEFT JOIN)

Faça junto. Missão 14. Mostre o titulo de TODAS as séries e o nome da editora (apelido editora), inclusive das séries sem editora.

O **LEFT JOIN** mantém todas as linhas da tabela da esquerda, mesmo sem par na direita. Onde não há editora, o resultado mostra **NULL**. Compare com a missão anterior: agora a Tower of God aparece.

```
SQL · missão 14
SELECT s.titulo, e.nome AS editora
FROM series s
LEFT JOIN editoras e ON s.editora_id = e.id;
```

RESULTADO ESPERADO

titulo	editora
Dragon Ball	Shueisha
One Piece	Shueisha
Naruto	Shueisha
Jujutsu Kaisen	Shueisha
Attack on Titan	Kodansha
Homem-Aranha	Marvel Comics
Turma da Mônica Jovem	Maurício de Sousa Produções
Tower of God	NULL

Pare e pense. Use JOIN quando só interessam os registros que têm par. Use LEFT JOIN quando precisa ver também quem não tem.

15 Estoque por série

Faça junto. Missão 15. Mostre o titulo de cada série, quantos volumes ela tem (apelido volumes) e o estoque total (apelido estoque).

GROUP BY junta as linhas em grupos, e as funções de agregação resumem cada grupo: **COUNT(*)** conta linhas e **SUM(coluna)** soma valores.

SQL · missão 15

```sql
SELECT s.titulo, COUNT(*) AS volumes, SUM(v.estoque) AS estoque
FROM series s
JOIN volumes v ON v.serie_id = s.id
GROUP BY s.titulo;
```

RESULTADO ESPERADO

titulo	volumes	estoque
Dragon Ball	3	20
One Piece	2	25
Naruto	2	25
Jujutsu Kaisen	1	9
Attack on Titan	1	6
Homem-Aranha	2	4
Turma da Mônica Jovem	1	18
Tower of God	2	10

16 Séries bem estocadas

Faça junto. Missão 16. Do resultado anterior, mantenha só as séries com estoque total maior que 15, do maior estoque para o menor.

HAVING filtra os **grupos** depois da agregação (WHERE filtra as linhas antes). Aqui só ficam as séries cujo estoque somado passa de 15.

SQL · missão 16

```sql
SELECT s.titulo, SUM(v.estoque) AS estoque
FROM series s
JOIN volumes v ON v.serie_id = s.id
GROUP BY s.titulo
HAVING SUM(v.estoque) > 15
ORDER BY estoque DESC;
```

RESULTADO ESPERADO

titulo	estoque
One Piece	25
Naruto	25
Dragon Ball	20
Turma da Mônica Jovem	18

Erro comum. Não dá para usar SUM no WHERE. Filtro sobre uma conta de grupo é sempre HAVING.

17 Situação do volume (CASE)

Faça junto. Missão 17. Mostre titulo da série, numero do volume e uma coluna situacao: 'esgotado' se o estoque é 0, 'poucas unidades' se é menor que 6, e 'disponível' nos demais casos.

CASE é o “se... então... senão” do SQL: testa cada WHEN em ordem e devolve o primeiro que for verdadeiro. Termina sempre com **END**.

SQL · missão 17

```sql
SELECT s.titulo, v.numero, CASE
    WHEN v.estoque = 0 THEN 'esgotado'
    WHEN v.estoque < 6 THEN 'poucas unidades'
    ELSE 'disponível'
    END AS situacao
FROM volumes v
JOIN series s ON s.id = v.serie_id;
```

RESULTADO ESPERADO

titulo	numero	situacao
Dragon Ball	1	disponível
Dragon Ball	2	disponível
Dragon Ball	3	esgotado
One Piece	1	disponível
One Piece	2	disponível
Naruto	1	disponível
Naruto	2	poucas unidades
Jujutsu Kaisen	1	disponível
Attack on Titan	1	disponível
Homem-Aranha	1	poucas unidades
Homem-Aranha	2	esgotado
Turma da Mônica Jovem	1	disponível

(mostradas as primeiras 12 linhas)

18 Rótulo e ano

Faça junto. Missão 18. Mostre uma coluna rotulo com o título em maiúsculas, um espaço, '#' e o número do volume (ex.: NARUTO #2), e uma coluna ano com o ano de lançamento.

Funções de texto e data: **UPPER** deixa em maiúsculas, **CONCAT** junta pedaços de texto e **YEAR** extrai o ano de uma data.

SQL · missão 18

```
SELECT CONCAT(UPPER(s.titulo), ' #', v.numero) AS rotulo, YEAR(v.lancamento) AS ano
FROM volumes v
JOIN series s ON s.id = v.serie_id;
```

RESULTADO ESPERADO

rotulo	ano
DRAGON BALL #1	2020
DRAGON BALL #2	2020
DRAGON BALL #3	2020
ONE PIECE #1	2021
ONE PIECE #2	2021
NARUTO #1	2019
NARUTO #2	2019
JUJUTSU KAISEN #1	2022
ATTACK ON TITAN #1	2021
HOMEM-ARANHA #1	2023
HOMEM-ARANHA #2	2023
TURMA DA MÔNICA JOVEM #1	2022

(mostradas as primeiras 12 linhas)

19 Acima da média

Faça junto. Missão 19. Mostre serie_id, numero e preco dos volumes com preço maior que a média de todos os volumes.

Uma **subconsulta** é um SELECT entre parênteses dentro de outro. O banco calcula primeiro a média e depois usa o valor no filtro.

SQL · missão 19

```
SELECT serie_id, numero, preco
FROM volumes
WHERE preco > (SELECT AVG(preco)
FROM volumes);
```

RESULTADO ESPERADO

serie_id	numero	preco
5	1	39.90
6	1	49.90
6	2	49.90
8	1	44.90
8	2	44.90

Resumo do capítulo

- WHERE filtra linhas; HAVING filtra grupos.
- JOIN só traz quem tem par; LEFT JOIN traz todos da esquerda.
- CASE faz decisões dentro do SELECT; subconsulta é um SELECT dentro de outro.

CAPÍTULO 4

Clientes e pedidos: relação de muitos para muitos

Modelar clientes, pedidos e itens, e entender a relação muitos para muitos.

4

VOCÊ VAI APRENDER

- ✓ Tabela de ligação (N:N)
- ✓ Chave primária composta
- ✓ CURRENT_TIMESTAMP e a palavra DEFAULT
- ✓ JOIN de várias tabelas

Uma loja precisa vender. Aqui nascem **clientes**, **pedidos** e **itens_pedido**. Quando terminar a missão 22, olhe o diagrama: troque entre **DER** e **MER** e veja como a mesma estrutura é desenhada de dois jeitos.

20 Tabela clientes

Faça junto. Missão 20. Crie a tabela clientes com: id (automático, chave primária), nome (texto de até 50, obrigatório), email (texto de até 80, único), cidade (texto de até 30) e criado_em (DATETIME com padrão CURRENT_TIMESTAMP, a hora do cadastro).

Fase dos pedidos: três tabelas novas. Primeiro **clientes**. A coluna **criado_em** usa **DEFAULT CURRENT_TIMESTAMP**, que grava automaticamente a data e a hora do cadastro.

SQL · missão 20

```sql
CREATE TABLE clientes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  email VARCHAR(80) UNIQUE,
  cidade VARCHAR(30),
  criado_em DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

21 Tabela pedidos

Faça junto. Missão 21. Crie a tabela pedidos com: id (automático, chave primária), cliente_id (inteiro, obrigatório, chave estrangeira de clientes), data_pedido (DATE) e status (ENUM com 'aberto', 'pago', 'enviado' e 'cancelado', padrão 'aberto').

A tabela **pedidos** pertence a um cliente (chave estrangeira cliente_id). O **status** é um ENUM com valor padrão 'aberto'. Repare que aqui não usamos ON DELETE: o padrão é **RESTRICT**, ou seja, o banco não deixa apagar um cliente que tem pedidos.

SQL · missão 21

```sql
CREATE TABLE pedidos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  cliente_id INT NOT NULL,
  data_pedido DATE,
  status ENUM('aberto','pago','enviado','cancelado') DEFAULT 'aberto',
  FOREIGN KEY (cliente_id) REFERENCES clientes(id)
);
```

22 Tabela itens_pedido

Faça junto. Missão 22. Crie a tabela itens_pedido, que liga pedidos e volumes: pedido_id (inteiro, chave estrangeira de pedidos com ON DELETE CASCADE), volume_id (inteiro, chave estrangeira de volumes) e quantidade (inteiro, padrão 1, com CHECK quantidade > 0). A chave primária é a dupla (pedido_id, volume_id).

Um pedido tem vários volumes, e um volume aparece em vários pedidos. Isso é uma relação **muitos para muitos (N:N)**, e o banco resolve com uma **tabela de ligação**. A chave primária de itens_pedido é composta: o **par** (pedido_id, volume_id) é único, e cada coluna do par também é chave estrangeira.

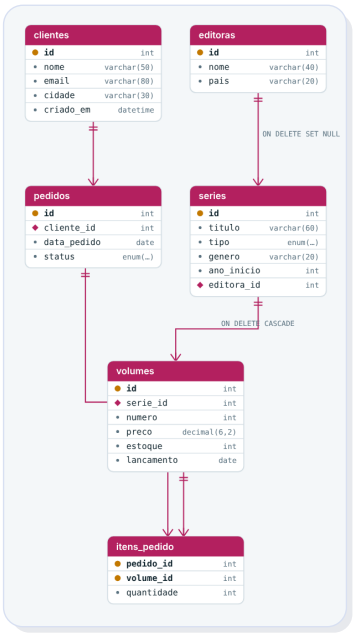
CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`PRIMARY KEY (pedido_id, volume_id)`	chave primária composta: o mesmo volume não entra duas vezes no mesmo pedido
`FOREIGN KEY (pedido_id) ... ON DELETE CASCADE`	apagou o pedido, os itens dele somem

SQL · missão 22

```sql
CREATE TABLE itens_pedido (
  pedido_id INT,
  volume_id INT,
  quantidade INT DEFAULT 1 CHECK (quantidade > 0),
  PRIMARY KEY (pedido_id, volume_id),
  FOREIGN KEY (pedido_id) REFERENCES pedidos(id) ON DELETE CASCADE,
  FOREIGN KEY (volume_id) REFERENCES volumes(id)
);
```

NO LAB



Importante. No **DER** a tabela de ligação aparece com duas setas. No **MER**, ela vira um losango com N dos dois lados: um relacionamento N para N. Alterne entre DER e MER no painel e compare.

23 Cadastre os clientes

Faça junto. Missão 23. Cadastre 4 clientes (o criado_em fica por conta do padrão): Ana Souza, ana@email.com, Recife · Bruno Lima, bruno@email.com, São Paulo · Carla Dias, carla@email.com, Recife · Diego Rocha, diego@email.com, Salvador.

Cadastre os clientes. O **criado_em** fica fora do INSERT: o DEFAULT cuida dele. Depois do INSERT, rode **SELECT * FROM clientes;** para ver a data preenchida.

SQL · missão 23

```sql
INSERT INTO clientes (nome, email, cidade) VALUES
  ('Ana Souza', 'ana@email.com', 'Recife'),
  ('Bruno Lima', 'bruno@email.com', 'São Paulo'),
  ('Carla Dias', 'carla@email.com', 'Recife'),
  ('Diego Rocha', 'diego@email.com', 'Salvador');
```

24 Cadastre os pedidos

Faça junto. Missão 24. Cadastre 5 pedidos (cliente_id, data_pedido, status): 1, 2024-03-01, pago · 1, 2024-03-15, enviado · 2, 2024-03-20, pago · 3, 2024-04-02, aberto (use a palavra DEFAULT no lugar do status) · 3, 2024-04-05, cancelado.

Na quarta linha, em vez de informar o status, usamos a palavra **DEFAULT**: ela pede ao banco que use o valor padrão da coluna (aqui, 'aberto').

SQL · missão 24

```sql
INSERT INTO pedidos (cliente_id, data_pedido, status) VALUES
  (1, '2024-03-01', 'pago'),
  (1, '2024-03-15', 'enviado'),
  (2, '2024-03-20', 'pago'),
  (3, '2024-04-02', DEFAULT),
  (3, '2024-04-05', 'cancelado');
```

25 Cadastre os itens

Faça junto. Missão 25. Cadastre os 9 itens abaixo (pedido_id, volume_id, quantidade).

DADOS PARA CADASTRAR

pedido	volume	quantidade
1	1	2
1	4	1
2	6	1
3	10	1
3	12	3
4	8	1
4	9	1
5	13	1
5	14	2

Os itens ligam pedidos a volumes. Cada par (pedido, volume) só pode aparecer uma vez, por causa da chave primária composta.

SQL · missão 25

```sql
INSERT INTO itens_pedido (pedido_id, volume_id, quantidade) VALUES
  (1, 1, 2), (1, 4, 1),
  (2, 6, 1),
  (3, 10, 1), (3, 12, 3),
  (4, 8, 1), (4, 9, 1),
  (5, 13, 1), (5, 14, 2);
```

Erro comum. Erro 1062 aqui quase sempre é um par repetido. Erro 1452 é um pedido ou volume que não existe.

26 Total de cada pedido

Faça junto. Missão 26. Mostre, para cada pedido, o id (apelido pedido), o nome do cliente e o total (apelido total), que é a soma de quantidade vezes preço dos itens.

Esta consulta junta **quatro tabelas**. Siga a cadeia: pedidos → clientes (quem comprou) e pedidos → itens_pedido → volumes (o que e quanto). A conta **quantidade * preco** é feita linha a linha e o SUM soma por pedido.

SQL · missão 26

```
SELECT p.id AS pedido, c.nome, SUM(i.quantidade * v.preco) AS total
FROM pedidos p
JOIN clientes c ON c.id = p.cliente_id
JOIN itens_pedido i ON i.pedido_id = p.id
JOIN volumes v ON v.id = i.volume_id
GROUP BY p.id, c.nome;
```

RESULTADO ESPERADO

pedido	nome	total
1	Ana Souza	92.70
2	Ana Souza	27.90
3	Bruno Lima	124.60
4	Carla Dias	74.80
5	Carla Dias	134.70

27 Quem nunca comprou

Faça junto. Missão 27. Mostre o nome dos clientes que nunca fizeram um pedido.

Para achar quem **não** tem par, use **LEFT JOIN** e filtre **IS NULL** na chave da outra tabela. Os clientes sem pedido ficam com NULL nessa coluna.

SQL · missão 27

```
SELECT c.nome
FROM clientes c
LEFT JOIN pedidos p ON p.cliente_id = c.id
WHERE p.id IS NULL;
```

RESULTADO ESPERADO

nome
Diego Rocha

28 Os mais vendidos

Faça junto. Missão 28. Mostre o titulo da série, o numero do volume e a soma das quantidades vendidas (apelido vendidos) dos 3 volumes mais vendidos.

ORDER BY ... DESC LIMIT 3 é a receita do “top 3”: ordena do maior para o menor e corta nas três primeiras linhas.

SQL · missão 28

```sql
SELECT s.titulo, v.numero, SUM(i.quantidade) AS vendidos
FROM itens_pedido i
JOIN volumes v ON v.id = i.volume_id
JOIN series s ON s.id = v.serie_id
GROUP BY s.titulo, v.numero
ORDER BY vendidos DESC
LIMIT 3;
```

RESULTADO ESPERADO

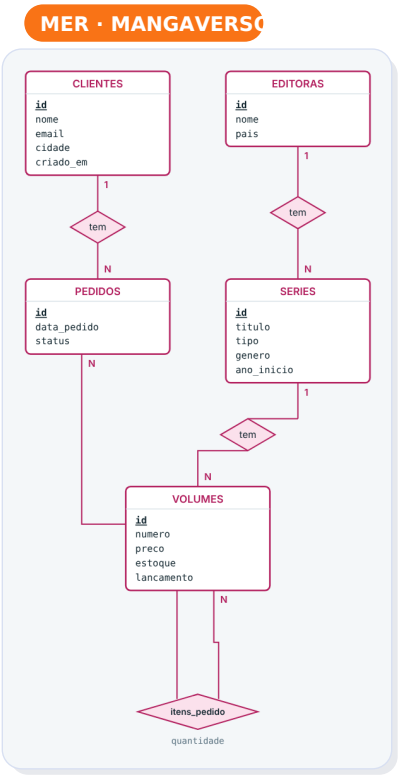
titulo	numero	vendidos
Turma da Mônica Jovem	1	3
Dragon Ball	1	2
Tower of God	2	2

Resumo do capítulo

- Relação N:N pede uma tabela de ligação com duas chaves estrangeiras.
- A chave primária composta impede o par repetido.
- Para achar quem não tem par: LEFT JOIN + IS NULL.

O mesmo desenho, em MER

O **MER** mostra o modelo conceitual: cada tabela vira uma **entidade**, cada chave estrangeira vira um **relacionamento** (1 para N) e a tabela de ligação vira um relacionamento **N para N**.



CAPÍTULO 5

Mudando com segurança

Alterar o banco com segurança: UPDATE, Modo seguro, ALTER TABLE, transações e efeitos de ON DELETE.

VOCÊ VAI APRENDER

- ✓ UPDATE e o erro 1175
- ✓ ALTER TABLE ADD COLUMN
- ✓ START TRANSACTION, ROLLBACK e COMMIT
- ✓ CASCADE e SET NULL na prática

Mudar dados é poderoso e perigoso. Este capítulo ensina a mudar com **cuidado**: ligar o Modo seguro, ensaiar numa transação e saber de antemão o que cada ON DELETE vai fazer.

29 Reajuste de preço

Faça junto. Missão 29. Reajuste em 10% o preço de todos os volumes de One Piece (serie_id = 2).

UPDATE muda dados que já existem. O **WHERE** escolhe quais linhas mudam; sem ele, **todas** mudam. Aqui, o preço é multiplicado por 1,10 só nos volumes de One Piece.

SQL · missão 29

```
UPDATE volumes SET preco = preco * 1.10 WHERE serie_id = 2;
```

Cuidado. Nunca rode UPDATE ou DELETE sem WHERE a não ser que queira mesmo mudar a tabela inteira. A próxima missão ensina como se proteger disso.

30 Ligue o Modo seguro

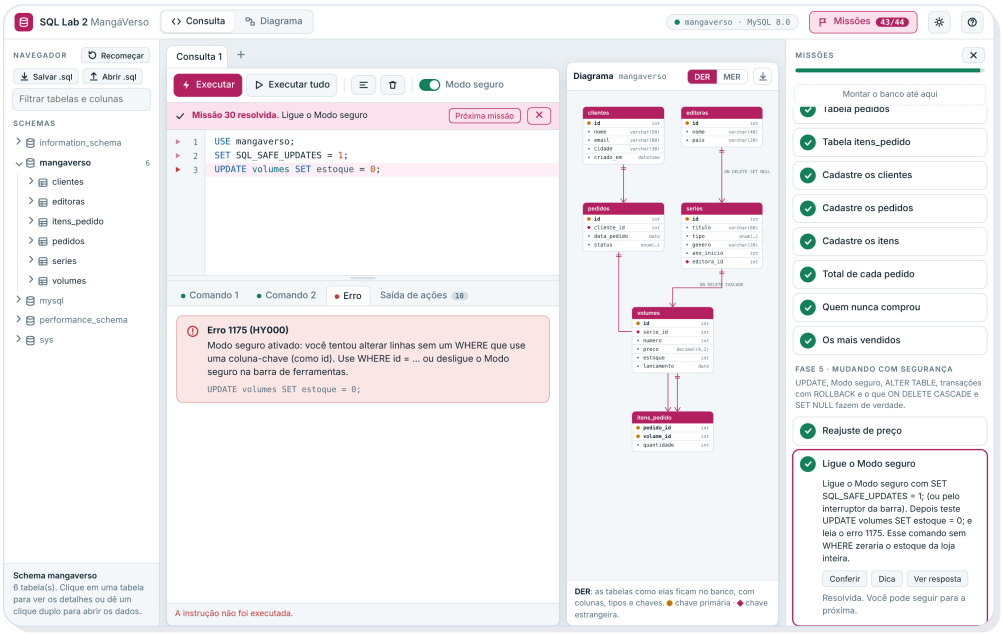
Faça junto. Missão 30. Ligue o Modo seguro com SET SQL_SAFE_UPDATES = 1; (ou pelo interruptor da barra). Depois teste UPDATE volumes SET estoque = 0; e leia o erro 1175. Esse comando sem WHERE zeraria o estoque da loja inteira.

O **Modo seguro** (SQL_SAFE_UPDATES) é um cinto de segurança: com ele ligado, o banco recusa UPDATE e DELETE sem WHERE em uma coluna-chave, com o **erro 1175**. Ligue-o e teste **UPDATE volumes SET estoque = 0;** para ler a mensagem. A partir daqui, deixe o modo seguro ligado.

```
SQL · missão 30

SET SQL_SAFE_UPDATES = 1;
```

NO LAB



Importante. Você também liga e desliga o modo seguro pelo interruptor **Modo seguro** da barra do editor. Com ele ligado, use sempre **WHERE id = ...** (ou outra coluna-chave).

31 Nova coluna: nota

Faça junto. Missão 31. Acrescente na tabela series a coluna nota, do tipo DECIMAL(3,1). Depois dê nota 9.5 à série de id 1.

ALTER TABLE ... ADD COLUMN acrescenta uma coluna a uma tabela que já existe. As linhas antigas recebem NULL nela. Depois, um UPDATE com WHERE na chave primária preenche o valor de uma série.

SQL · missão 31

```
ALTER TABLE series ADD COLUMN nota DECIMAL(3,1);
UPDATE series SET nota = 9.5 WHERE id = 1;
```

32 Ensaio com ROLLBACK

Faça junto. Missão 32. Abra uma transação, apague todos os itens do pedido 5 e desfaça tudo com ROLLBACK. No final os 9 itens precisam continuar lá.

Uma **transação** agrupa comandos para valer tudo ou nada. **START TRANSACTION** abre; **ROLLBACK** desfaz tudo o que veio depois; **COMMIT** confirma de vez. Aqui você apaga os itens do pedido 5 e, em seguida, desfaz. Confira com **SELECT COUNT(*) FROM itens_pedido;**: continuam 9.

CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`START TRANSACTION;`	começa o “ensaio”
`DELETE FROM itens_pedido WHERE pedido_id = 5;`	apaga, mas ainda de teste
`ROLLBACK;`	volta tudo ao que era antes

SQL · missão 32

```
START TRANSACTION;
DELETE FROM itens_pedido WHERE pedido_id = 5;
ROLLBACK;
```

33 Apagando em cascata

Faça junto. Missão 33. Apague de vez o pedido 5. Como a chave estrangeira de itens_pedido tem ON DELETE CASCADE, os itens dele somem sozinhos.

Apagar um pedido com **DELETE** remove também os itens dele, por causa do **ON DELETE CASCADE** que você declarou na tabela itens_pedido. Confira com um SELECT em itens_pedido.

SQL · missão 33

```
DELETE FROM pedidos WHERE id = 5;
```

34 Apagando com SET NULL

Faça junto. Missão 34. Apague a editora Kodansha (id 2). A série Attack on Titan não some: a editora_id dela vira NULL (ON DELETE SET NULL).

Aqui o efeito é diferente: apagar a editora Kodansha **não** apaga a série. O **ON DELETE SET NULL** da tabela series faz o editora_id de Attack on Titan virar NULL. Rode **SELECT titulo, editora_id FROM series;** para ver.

SQL · missão 34

```sql
DELETE FROM editoras WHERE id = 2;
```

Resumo do capítulo

- UPDATE e DELETE sempre com WHERE. O Modo seguro ajuda a lembrar.
- ROLLBACK desfaz o que a transação fez; COMMIT confirma.
- CASCADE apaga os filhos; SET NULL só solta o vínculo.

CAPÍTULO 6

Relatórios guardados: Views e UNION

Guardar consultas com nome (VIEW) e juntar listas com UNION.

VOCÊ VAI APRENDER

- ✓ CREATE VIEW
- ✓ Consultar uma view como tabela
- ✓ UNION e UNION ALL

Relatórios que você usa toda hora merecem um nome. Uma **view** guarda a consulta; **UNION** junta resultados de consultas diferentes numa lista só.

35 View do catálogo

Faça junto. Missão 35. Crie a view vw_catalogo com: titulo (da série), editora (nome da editora, pode ficar NULL), numero, preco e estoque de cada volume.

Uma **VIEW** é uma consulta guardada com um nome. Ela não copia dados: toda vez que você a consulta, o banco refaz o SELECT por trás. É ótima para relatórios que você usa sempre.

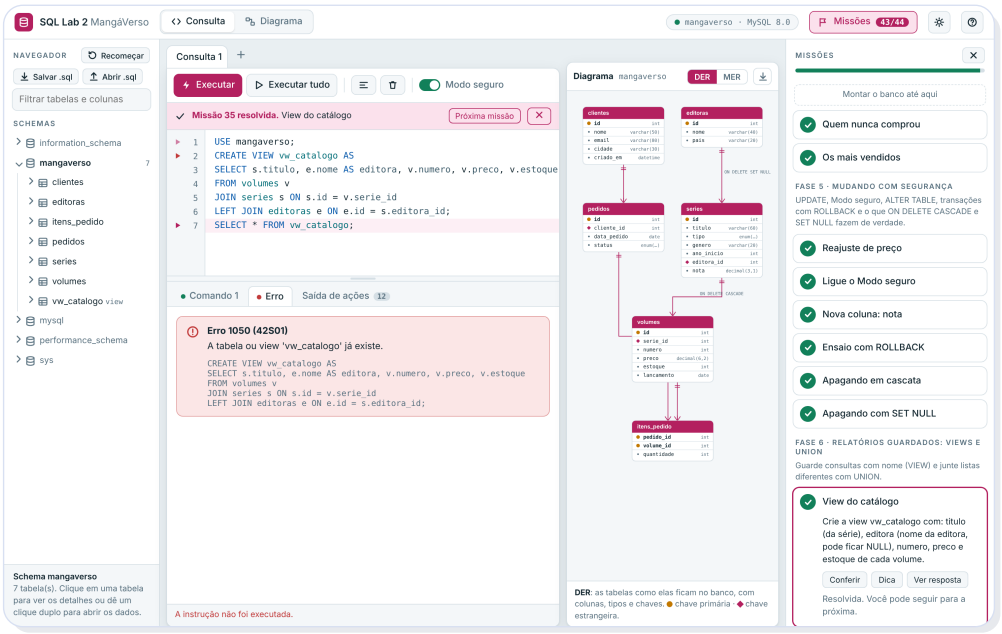
CADA PEDAÇO DO COMANDO

Pedaço	O que significa
`CREATE VIEW vw_catalogo AS`	dá o nome vw_catalogo à consulta
`LEFT JOIN editoras`	mantém séries sem editora

SQL · missão 35

```sql
CREATE VIEW vw_catalogo AS
SELECT s.titulo, e.nome AS editora, v.numero, v.preco, v.estoque
FROM volumes v
JOIN series s ON s.id = v.serie_id
LEFT JOIN editoras e ON e.id = s.editora_id;
```

NO LAB



Dica. A view aparece no navegador da esquerda com o selo *view*. Convenção útil: comece o nome com **vw_**.

36 Consultando a view

Faça junto. Missão 36. Usando a view, mostre titulo e numero dos volumes esgotados (estoque igual a 0).

Agora você consulta a view como se fosse uma tabela. O JOIN complicado ficou escondido dentro dela.

```
SQL · missão 36

SELECT titulo, numero FROM vw_catalogo WHERE estoque = 0;
```

RESULTADO ESPERADO

titulo	numero
Dragon Ball	3
Homem-Aranha	2

37 View de totais

Faça junto. Missão 37. Crie a view vw_total_pedidos com pedido (id do pedido), cliente (nome) e total (soma de quantidade vezes preço).

Outra view, agora com agregação. Como ela lê as tabelas em tempo real, se um pedido mudar, o total muda junto.

SQL · missão 37

```sql
CREATE VIEW vw_total_pedidos AS
SELECT p.id AS pedido, c.nome AS cliente, SUM(i.quantidade * v.preco) AS total
FROM pedidos p
JOIN clientes c ON c.id = p.cliente_id
JOIN itens_pedido i ON i.pedido_id = p.id
JOIN volumes v ON v.id = i.volume_id
GROUP BY p.id, c.nome;
```

38 Maior pedido

Faça junto. Missão 38. Usando a view vw_total_pedidos, mostre cliente e total do pedido de maior valor.

Consultar uma view dentro de outra consulta deixa o SQL curto e legível: **ORDER BY total DESC LIMIT 1** devolve o maior.

SQL · missão 38

```sql
SELECT cliente, total FROM vw_total_pedidos ORDER BY total DESC LIMIT 1;
```

RESULTADO ESPERADO

cliente	total
Bruno Lima	124.60

39 Lista única com UNION

Faça junto. Missão 39. Faça uma lista única, em ordem alfabética, com os títulos das séries e os nomes das editoras: uma coluna nome e uma coluna origem ('série' ou 'editora').

UNION empilha o resultado de dois SELECTs em uma única lista, tirando repetidos. As duas consultas precisam ter o **mesmo número de colunas**; os nomes vêm do primeiro SELECT. O **ORDER BY** final vale para a lista toda. Para manter repetidos, use **UNION ALL**.

SQL · missão 39

```sql
SELECT titulo AS nome, 'série' AS origem FROM series
UNION
SELECT nome, 'editora' FROM editoras
ORDER BY nome;
```

RESULTADO ESPERADO

nome	origem
Attack on Titan	série
Dragon Ball	série
Homem-Aranha	série
Jujutsu Kaisen	série
Marvel Comics	editora
Maurício de Sousa Produções	editora
Naruto	série
One Piece	série
Shueisha	editora
Tower of God	série
Turma da Mônica Jovem	série

Resumo do capítulo

- Uma view é uma consulta guardada, sempre atualizada.
- UNION precisa do mesmo número de colunas e tira repetidos; UNION ALL mantém.

CAPÍTULO 7

Desafio final: monte o seu banco

Projetar e montar um banco inteiro sozinho, do CREATE DATABASE à view.

VOCÊ VAI APRENDER

- ✓ Planejar tabelas e relacionamentos
- ✓ Aplicar tudo o que aprendeu
- ✓ Conferir o DER e o MER do seu projeto

Última fase, sem muleta: crie o **clube_leitores**. As missões dizem *o que* criar; o *como* você já sabe. Se travar, use as dicas do painel.

40 Crie o clube

Faça junto. Missão 40. Crie o banco clube_leitores e deixe-o em uso.

Desafio final! Agora você cria um banco inteiro sozinho: o **clube_leitores**, onde leitores escrevem resenhas sobre séries. Comece como na missão 1.

SQL · missão 40

```
CREATE DATABASE clube_leitores;
USE clube_leitores;
```

41 Tabela leitores

Faça junto. Missão 41. Crie a tabela leitores com: id (automático, chave primária), nome (texto, obrigatório), email (texto, único) e cidade (texto, opcional).

Primeira tabela do clube. Use o que aprendeu: chave primária automática, NOT NULL, UNIQUE.

```sql
SQL · missão 41

CREATE TABLE leitores (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  email VARCHAR(80) UNIQUE,
  cidade VARCHAR(30)
);
```

42 Tabela resenhas

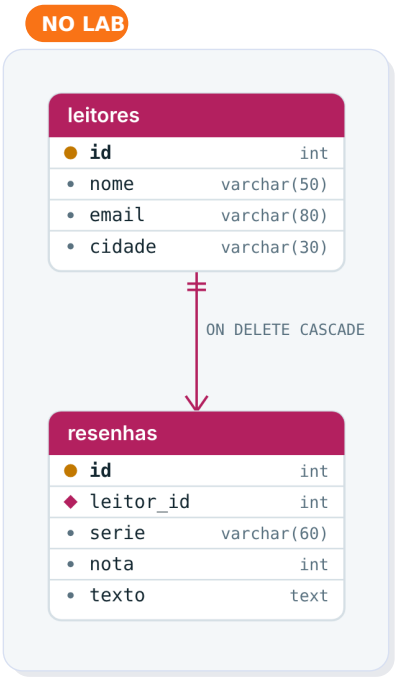
Faça junto. Missão 42. Crie a tabela resenhas com: id (automático, chave primária), leitor_id (inteiro, obrigatório, chave estrangeira de leitores com ON DELETE CASCADE), serie (texto), nota (inteiro com CHECK entre 1 e 5) e texto (TEXT).

A tabela **resenhas** pertence a um leitor. Use a chave estrangeira com **ON DELETE CASCADE** (sem leitor, a resenha não faz sentido) e um **CHECK** para a nota ficar entre 1 e 5.

```sql
SQL · missão 42

CREATE TABLE resenhas (
  id INT AUTO_INCREMENT PRIMARY KEY,
  leitor_id INT NOT NULL,
  serie VARCHAR(60),
  nota INT CHECK (nota BETWEEN 1 AND 5),
  texto TEXT,
  FOREIGN KEY (leitor_id) REFERENCES leitores(id) ON DELETE CASCADE
);
```

NO LAB



Importante. Olhe o diagrama: você acaba de criar um banco novo e o desenho dele nasceu sozinho. Compare o **DER** e o **MER** do clube.

43 Cadastre os dados

Faça junto. Missão 43. Cadastre pelo menos 3 leitores e 5 resenhas, de séries variadas, com notas de 1 a 5. Invente à vontade!

Cadastre os leitores **primeiro** e as resenhas depois: as resenhas dependem do id dos leitores. Invente à vontade, só respeite as notas de 1 a 5.

SQL · missão 43

```sql
INSERT INTO leitores (nome, email, cidade) VALUES
  ('Lia', 'lia@email.com', 'Recife'),
  ('Davi', 'davi@email.com', 'Natal'),
  ('Mel', 'mel@email.com', 'Olinda');
INSERT INTO resenhas (leitor_id, serie, nota, texto) VALUES
  (1, 'Naruto', 5, 'Emocionante do começo ao fim.'),
  (1, 'One Piece', 5, 'Aventura sem fim.'),
  (2, 'Naruto', 4, 'Ótimas lutas.'),
  (2, 'Homem-Aranha', 3, 'Gostei, mas esperava mais.'),
  (3, 'One Piece', 4, 'Personagens incríveis.');
```

44 Sua própria view

Faça junto. Missão 44. Crie a view vw_media_notas com a serie, a média das notas (apelido media) e a quantidade de resenhas (apelido resenhas) de cada série.

Feche com uma view própria: a média das notas e a quantidade de resenhas de cada série. Depois consulte-a com **SELECT * FROM vw_media_notas;**.

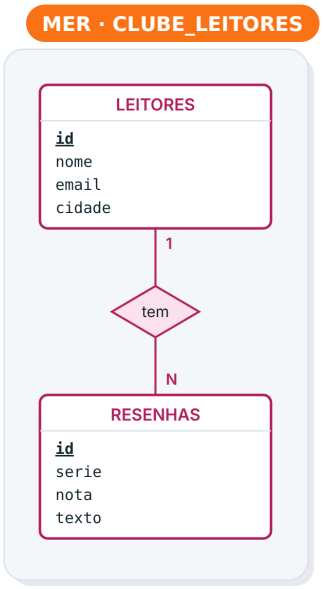
SQL · missão 44

```sql
CREATE VIEW vw_media_notas AS
SELECT serie, ROUND(AVG(nota), 1) AS media, COUNT(*) AS resenhas
FROM resenhas
GROUP BY serie;
```

Resumo do capítulo

- Um banco bem pensado começa pelas tabelas “pai” e termina nas “filhas”.
- O diagrama é a sua conferência visual: se a linha não aparece, falta a chave estrangeira.

O diagrama do seu clube



APÊNDICE

Cola de comandos

Todos os comandos novos do Lab 2 em uma página.

VOCÊ VAI APRENDER

- ✓ Consulta rápida
- ✓ Quando usar cada comando

Comando	Para que serve
`CREATE DATABASE nome;`	cria um banco
`USE nome;`	escolhe o banco de trabalho
`ENUM('a','b')`	coluna com valores de uma lista fechada
`DEFAULT valor`	valor usado quando nada é informado
`UNIQUE (a, b) / CHECK (cond)`	impede repetir a combinação / regra do dado
`FOREIGN KEY (c) REFERENCES t(id)`	liga a coluna c à chave de outra tabela
`ON DELETE CASCADE`	apaga os filhos junto com o pai
`ON DELETE SET NULL`	solta o vínculo (vira NULL) ao apagar o pai
`PRIMARY KEY (a, b)`	chave primária composta
`INSERT ... VALUES (..), (..)`	várias linhas de uma vez
`LEFT JOIN`	mantém todas as linhas da esquerda
`CASE WHEN ... END`	decisão dentro do SELECT
`HAVING / (SELECT ...)`	filtra grupos / subconsulta
`SET SQL_SAFE_UPDATES = 1;`	liga o Modo seguro
`ALTER TABLE t ADD COLUMN c tipo;`	acrescenta coluna
`START TRANSACTION / ROLLBACK / COMMIT`	ensaiar e desfazer ou confirmar
`CREATE VIEW v AS SELECT ...`	guarda uma consulta com nome
`SELECT ... UNION SELECT ...`	junta listas, sem repetidos

APÊNDICE

Lendo o MER e o DER

Como interpretar os dois desenhos do painel ao lado do editor.

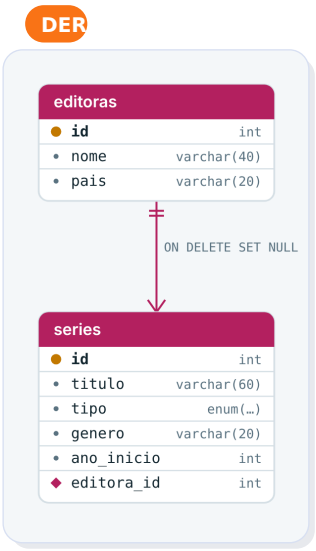
VOCÊ VAI APRENDER

- ✓ Símbolos do DER
- ✓ Símbolos do MER
- ✓ Do DER para o MER

DER (Diagrama de Entidade e Relacionamento, versão de tabelas)

É o desenho que mais se aproxima do banco real: uma caixa por tabela, com todas as colunas e seus tipos.

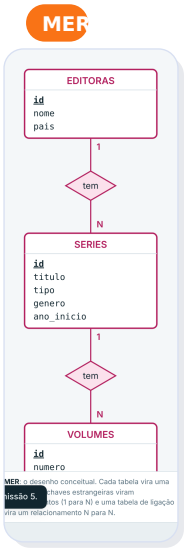
- **Bolinha dourada**: chave primária.
- **Losango rosa**: chave estrangeira.
- **Linha com pé de galinha**: o lado “muitos” da relação.
- **Legenda ON DELETE**: o que acontece ao apagar o registro pai.



MER (Modelo de Entidade e Relacionamento, versão conceitual)

Esconde os detalhes técnicos e mostra o **significado**: entidades (retângulos), relacionamentos (losangos) e atributos (os nomes dentro da caixa, com a chave primária sublinhada). Os números 1 e N indicam a cardinalidade.

- Uma chave estrangeira vira um relacionamento **1 para N**.
- Uma tabela de ligação (duas chaves estrangeiras na chave primária) vira um relacionamento **N para N**.



APÊNDICE

Erros comuns e o que fazer

Mensagens do banco que você vai encontrar e como resolvê-las.

VOCÊ VAI APRENDER

- ✓ Ler a mensagem de erro
- ✓ Corrigir com calma

Mensagem	Causa provável	Como resolver
`1050 Table already exists`	a tabela já foi criada	use outro nome ou DROP TABLE antes
`1062 Duplicate entry`	valor repetido em coluna UNIQUE ou chave primária	troque o valor ou cadastre só uma vez
`1452 Cannot add or update a child row`	chave estrangeira aponta para um pai que não existe	cadastre o pai antes
`1451 Cannot delete a parent row`	o pai ainda tem filhos (RESTRICT)	apague os filhos antes ou use CASCADE
`1175 Safe update mode`	UPDATE ou DELETE sem WHERE na chave	adicione WHERE id = ...
`3819 Check constraint violated`	valor fora da regra do CHECK	use um valor válido
`1064 Syntax error`	erro de escrita (vírgula, parêntese, aspas)	releia o comando com calma
`1046 No database selected`	faltou o USE	rode USE nome_do_banco;

Dica. Erros fazem parte do aprendizado. Leia a mensagem inteira: quase sempre ela diz o número da linha e o que o banco não entendeu.